

Menú Selector de la Aplicación

El proyecto está diseñado bajo un único **Menú Principal**, centralizando todos los apartados para no tener que ejecutarlos por consola individualmente. Este menú inicializa y despliega de manera ordenada las interfaces especializadas para cada uno de los 5 ejercicios de esta actividad.

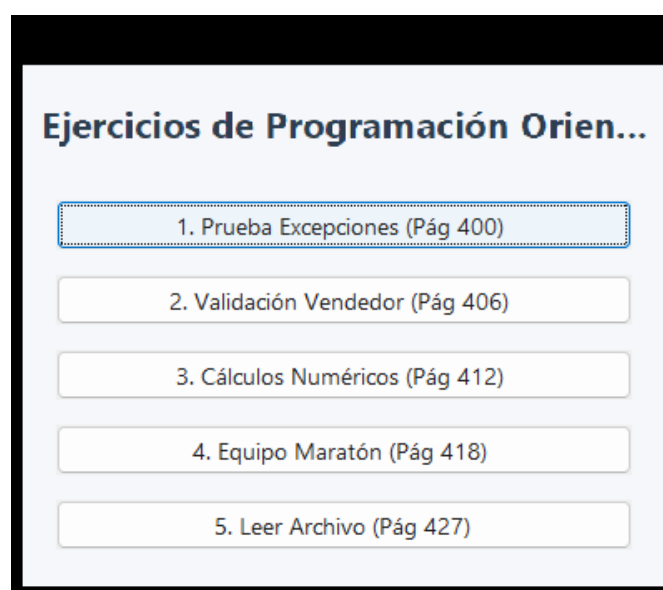


Figura 1: El menú principal que unifica e integra la Actividad 4.

1. Prueba de Excepciones (Pág. 400)

Requerimientos técnicos del ejercicio (Enunciado):

Clase PruebaExcepciones: ¿Cuál es el resultado de la ejecución del método main del programa? Determinar qué se imprime en pantalla. El programa debe contener un bloque principal con dos bloques try que generan excepciones. El primero genera una `ArithmeticException` mediante una división por cero (`10000/0`), y el segundo genera una `Exception` invocando un método `toString()` sobre un objeto instanciado como nulo. En ambos casos, se deben estructurar los bloques catch y finally y verificar el flujo de impresión.

1.1. Interfaz de usuario

A continuación se detalla la interfaz gráfica construida para la ejecución y visualización de la traza de excepciones en lugar de usar solo la consola. Se provee un botón para iniciar la simulación del error y un área de texto estilizada que muestra paso a paso las capturas de error y finalizadores.

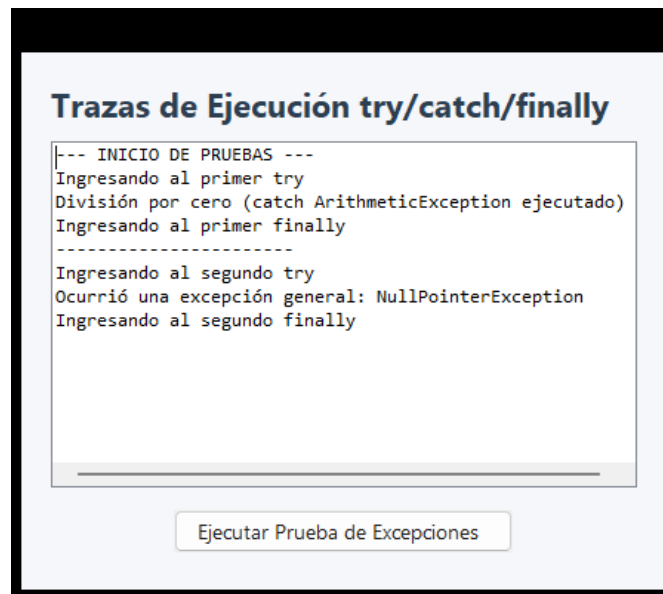


Figura 2: Ventana de visualización de Trazas de Excepciones tras la ejecución de las pruebas.

1.2. Diagrama de clases

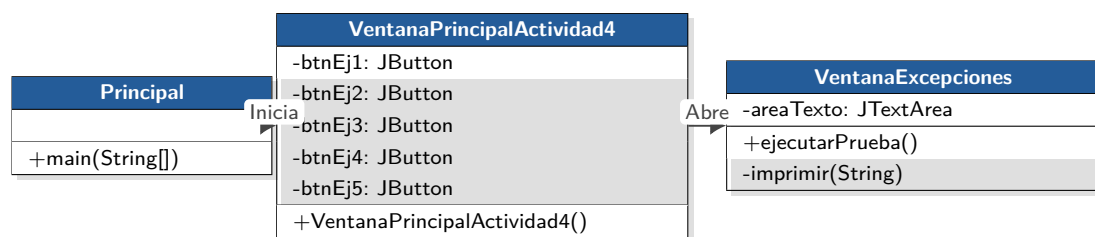


Figura 3: Diagrama de clases para el ejercicio de Prueba de Excepciones.

1.3. Casos de uso

■ Caso de Uso 1: Ejecutar Trazas de Excepción

- **Actor:** Usuario.
- **Precondición:** La ventana *Prueba Excepciones* debe estar desplegada.
- **Flujo Principal:** El usuario presiona el botón *Ejecutar Prueba de Excepciones*. El sistema ejecuta internamente la división entera por cero y la referencia nula, captura las excepciones correspondientes (*ArithmeticException* y *Exception*) e imprime en pantalla el flujo paso a paso incluyendo las sentencias obligatorias de los bloques *finally*.

1.4. Link

El código correspondiente a este ejercicio puede verificarse en: https://github.com/mateolikescats/P00/tree/main/Actividad_4/src/PruebaExcepciones

2. Validación Vendedor (Pág. 406)

Requerimientos técnicos del ejercicio (Enunciado):

Clase Vendedor: Se requiere implementar una clase Vendedor que posea los siguientes atributos: nombre (tipo String), apellidos (tipo String) y edad (tipo int). La clase contiene un constructor que inicialice los atributos. Además, la clase posee los métodos:

- **Imprimir:** muestra por pantalla los valores de sus atributos.
- **Verificar edad:** recibe como parámetro un valor entero que representa la edad del vendedor. Para que pueda desempeñar sus labores se requiere que sea mayor de edad (mayor de 18 años). Si esta condición no se cumple, se lanza una excepción de tipo `IllegalArgumentException` con el mensaje “El vendedor debe ser mayor de 18 años”. Además, se evalúa si la edad se encuentra en el rango de 0 a 120, si no se cumple, se genera otra excepción de tipo `IllegalArgumentException` con el mensaje “La edad no puede ser negativa ni mayor a 120”. Si la edad cumple estos requerimientos se puede instanciar el objeto.

Además, se requiere que los datos del vendedor se ingresen por teclado (en nuestro caso, a través de una GUI).

2.1. Interfaz de usuario

La GUI para la creación del vendedor cuenta con campos para ingresar el nombre, los apellidos y la edad del vendedor. Al pulsar el botón “Crear y Validar”, se efectúa la validación solicitada y, si es correcta, se notifica el éxito. Si la edad está fuera de los rangos válidos, se captura la excepción correspondiente y se muestra en una alerta.

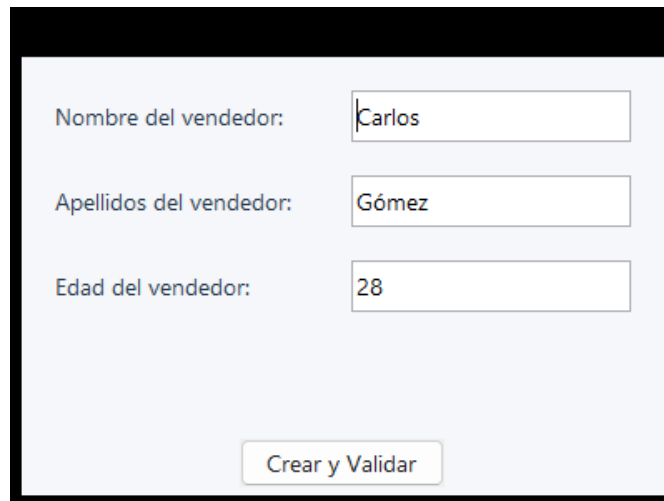


Figura 4: Ventana de registro y validación del Vendedor.

2.2. Diagrama de clases

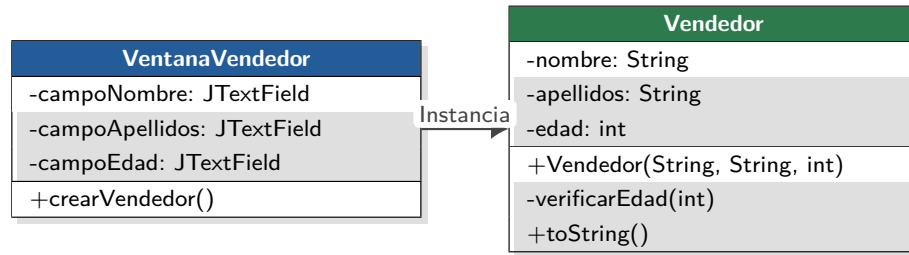


Figura 5: Diagrama de clases para el ejercicio de Validación del Vendedor.

2.3. Casos de uso

■ Caso de Uso 1: Crear Vendedor con Datos Válidos

- **Actor:** Usuario.
- **Flujo Principal:** El usuario digita el nombre, los apellidos y una edad dentro del rango [18, 120] y hace clic en *Crear y Validar*. El sistema valida e instancia el Vendedor, mostrando los datos con el método `toString()`.

■ Caso de Uso 2: Intentar Crear Vendedor Menor de Edad o Edad Fuera de Rango

- **Actor:** Usuario.
- **Flujo Alternativo:** El usuario ingresa una edad menor de 18 años, negativa, o mayor a 120. El sistema interrumpe la lógica lanzando `IllegalArgumentException` y el controlador notifica el error formal en pantalla.

2.4. Link

El código correspondiente a este ejercicio puede verificarse en: https://github.com/mateolikescats/P00/tree/main/Actividad_4/src/Vendedor

3. Cálculos Numéricos (Pág. 412)

Requerimientos técnicos del ejercicio (Enunciado):

Clase CálculosNuméricos: Se requiere definir una clase que realice las siguientes operaciones:

- Calcular el **logaritmo neperiano** recibiendo un valor `double` como parámetro. Este método debe ser estático. Si el valor no es positivo se genera una excepción aritmética.
- Calcular la **raíz cuadrada** recibiendo un valor `double` como parámetro. Este método debe ser estático. Si el valor no es positivo se genera una excepción aritmética.
- Se debe crear un método `main` que utilice dichos métodos ingresando un valor por teclado.

3.1. Interfaz de usuario

La interfaz gráfica de usuario permite la captura del valor numérico e incorpora dos botones autónomos para activar por separado los cálculos de logaritmo y raíz cuadrada. El resultado o error aritmético se muestra en la etiqueta de resultados o como alerta interactiva.

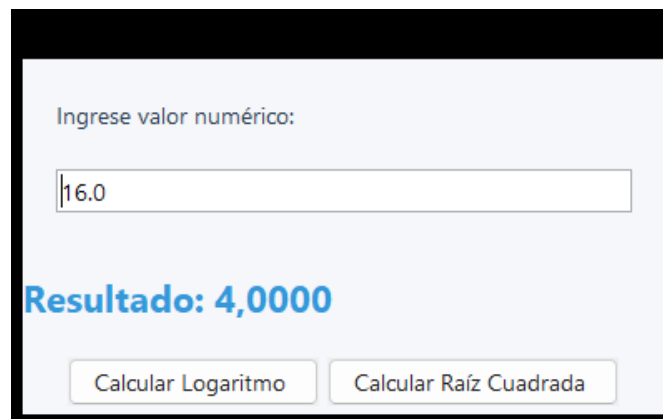


Figura 6: Ventana para realizar Cálculos Numéricos avanzados con control de excepciones.

3.2. Diagrama de clases

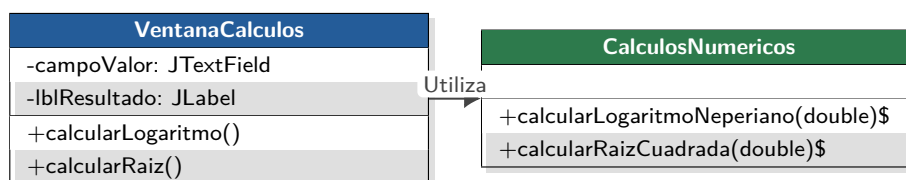


Figura 7: Diagrama de clases para el ejercicio de Cálculos Numéricos.

3.3. Casos de uso

- Caso de Uso 1: Calcular Operación con Valor Positivo
 - Actor: Usuario.

- **Flujo Principal:** El usuario escribe un número real positivo y presiona cualquiera de los botones de cálculo. El sistema computa la raíz o el logaritmo de forma estática y muestra el resultado en el label inferior.
- **Caso de Uso 2: Gestionar Excepciones Numéricas**
 - **Actor:** Usuario.
 - **Flujo Alternativo:** Si el número ingresado es negativo o posee texto, el sistema lanza y captura una `ArithmeticException` o un `NumberFormatException`, evitando el colapso del sistema y alertando al usuario visualmente.

3.4. Link

El código correspondiente a este ejercicio puede verificarse en: https://github.com/mateolikescats/P00/tree/main/Actividad_4/src/CalculosNumericos

4. Equipo Maratón (Pág. 418)

Requerimientos técnicos del ejercicio (Enunciado):

Clase EquipoMaratónProgramación: Un equipo desea participar en una maratón. Tiene los atributos: Nombre del equipo, Universidad, Lenguaje de programación, y Tamaño del equipo. El equipo está conformado por varios programadores (mínimo dos y máximo tres), cada uno con nombre y apellidos. Se requieren los siguientes métodos:

- Un método para determinar si el equipo está completo.
- Un método para añadir programadores al equipo. Si el equipo está lleno se debe imprimir la excepción correspondiente.
- Un método para validar los atributos nombre y apellidos de un programador para que reciban **solo texto**. Si se reciben datos numéricos se debe generar la excepción correspondiente. Además, no se permiten campos con una longitud igual o superior a 20 caracteres.

Ejercicios propuestos (Pág 425): Como adición formativa en la actividad, el libro propone desarrollar también una validación estricta de contraseña (mínimo 8 caracteres, sin espacios, con mayúsculas, un número y un carácter especial, capturando los errores mediante excepciones). En nuestra interfaz, las aserciones sobre atributos cubren estos lineamientos aplicando lógica de validación similar sobre los nombres de los miembros del equipo.

4.1. Interfaz de usuario

La ventana está dividida en dos etapas secuenciales: primero se crea la estructura del equipo y luego se habilitan los campos para ir agregando iterativamente cada programador a la lista, comprobando caracteres ilícitos y el desbordamiento de capacidad.

The screenshot shows a user interface for managing a marathon team. It is divided into three main sections. The top section, titled 'Datos del Equipo', contains three input fields: 'Nombre del Equipo' (with the value 'CodeCats'), 'Universidad' (with the value 'UNAL'), and 'Lenguaje' (with the value 'Java'). Below these fields is a 'Crear Equipo' button. The middle section, titled 'Añadir Programador', contains two input fields: 'Nombre' and 'Apellidos'. Below these fields is an 'Añadir Programador' button. The bottom section is a list box containing the names of the team members: 'Mateo Quiceno' and 'Juan Pérez'.

Figura 8: Ventana de gestión y registro de miembros del Equipo Maratón.

4.2. Diagrama de clases

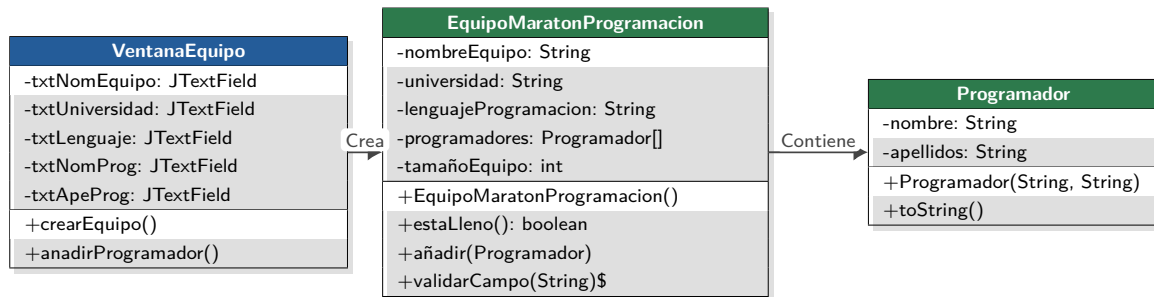


Figura 9: Diagrama de clases para el ejercicio de Equipo Maratón de Programación.

4.3. Casos de uso

■ Caso de Uso 1: Crear Equipo e Inscribir Integrantes

- **Actor:** Usuario.
- **Flujo Principal:** El usuario inicializa el equipo y añade hasta 3 programadores. El sistema valida las longitudes (≤ 20) y la ausencia de dígitos en los nombres, lanzando excepciones si no cumplen la política estructural.

■ Caso de Uso 2: Controlar Exceso de Capacidad

- **Actor:** Sistema.
- **Flujo Alternativo:** Intentar añadir un 4to integrante ocasiona que `añadir()` genere una excepción de capacidad agotada, impidiendo la modificación del arreglo interno.

4.4. Link

El código correspondiente a este ejercicio puede verificarse en: https://github.com/mateolikescats/P00/tree/main/Actividad_4/src/EquipoMaraton

5. Leer Archivo (Pág. 427)

Requerimientos técnicos del ejercicio (Enunciado):

Clase LeerArchivo: Se tiene un archivo de texto denominado prueba.txt en una cierta localización en un sistema de archivos. Se requiere desarrollar un programa que lea dicho archivo de texto utilizando un flujo de bytes que muestre los contenidos del archivo en pantalla. Para esto, se debe instanciar un flujo con `FileInputStream`, enlazarlo con `InputStreamReader` y finalmente utilizar `BufferedReader` para efectuar la lectura con `readLine()` asegurando la captura correcta de `IOException` si el documento es inaccesible o si el flujo falla.

5.1. Interfaz de usuario

La ventana del lector cuenta con un selector nativo de Windows (`JFileChooser`) activado por botón. El contenido se carga dinámicamente y se imprime en el área de texto protegida.



Figura 10: Ventana del Lector de Archivos de Texto con control de fallos I/O.

5.2. Diagrama de clases

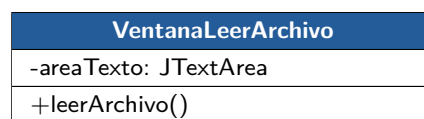


Figura 11: Diagrama de clases para el ejercicio de Lectura de Archivos.

5.3. Casos de uso

■ Caso de Uso 1: Seleccionar y Leer Archivo de Texto

- **Actor:** Usuario.
- **Flujo Principal:** El usuario selecciona un archivo .txt a través de la ventana de diálogo, el buffer extrae el contenido de bytes traduciéndolo a Unicode y este se renderiza línea por línea en pantalla.

■ Caso de Uso 2: Fallo de Lectura (IOException)

- **Actor:** Sistema.
- **Flujo Alternativo:** Si el flujo falla por inaccesibilidad o daño del disco, el sistema detecta `IOException` y advierte al usuario de manera controlada.

5.4. Link

El código correspondiente a este ejercicio puede verificarse en: https://github.com/mateolikescats/P00/tree/main/Actividad_4/src/LeerArchivo

6. Conclusión

Esta actividad sirvió para entender a fondo el ciclo de vida de los errores del sistema y la captura inteligente de excepciones en Java, delegando la responsabilidad de procesamiento desde las clases de negocio puras hacia los controladores gráficos de las ventanas. El uso de bloques `try-catch-finally` y el lanzamiento explícito de excepciones de usuario permiten modelar restricciones y resguardar la estabilidad del software, evitando caídas abruptas y transformando condiciones críticas de error en notificaciones dinámicas de cara al usuario final. Adicionalmente, se logró implementar correctamente arquitecturas de IO para lectura segura de archivos.